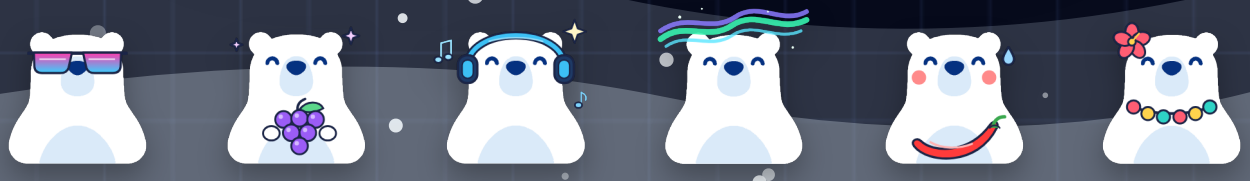
nanuq

TECHNICAL MANUAL

How Nanuq is built

Architecture with diagrams, each platform inside out and, at the end, every function in the code.





Version 1.46.1 · November 9, 2026 · English · Free software (GPL-3.0-or-later)



Contents

Part 1 explains the design; part 2 is the reference for every function.

ARCHITECTURE

1 **Overview** 6
One interface, three wrappers.

2 **The repository** 8
What lives in each folder.

3 **The page** 9
How app.js starts, navigates and reacts.

4 **Data and storage** 11
Anything about you is encrypted.

5 **Playback** 13
Where each song comes from and what happens when something fails.

6 **Library and Jellyfin** 15
The API Nanuq uses and what it builds on its own.

7 **DJ Nanuq** 16
Rule-based rounds, no AI.

8 **My devices** 17
One Nanuq plays and the others control it, through Jellyfin.

9 **The party** 18
Phones sending songs to a computer over the local network.

10 **The Android app** 20
WebView + native Java.

11 **The desktop app** 21
Electron for Linux and Windows.

12 **Security** 22
Nothing from outside is trusted.

13 **Building and releasing** 23
One command for every platform.

14 **Conventions and pitfalls** 24
What to know before changing anything.

15 **License** 25
GPLv3 or later, file by file.

DEEP LEVEL: EVERY FUNCTION

16 **web/app.js** 28
598 functions, in file order.

Base: helpers and storage 28
Server addresses 29
Jellyfin API 29
Audio: waveforms and synthesizer 31
Demo library 31
Settings and themes 33
Downloads (Android) 33
State and routing 34
Albums from tags 34
Lyrics search 35
Rendering helpers 35
Views 35
DJ Nanuq 35
Your year and Your week (video) 38
Song requests (SomeDL) 39
Player 40
Lyrics 42
Queue 43
Toasts, menus and dialogs 44
My devices 44
Party (watch party) 46
Global actions 48
Login 48

17 **Rest of the page** 49
Languages and service worker.

web/i18n.js 49

web/sw.js 49

18 **Android** 50
Native Java and the JavaScript bridge.

android/app/src/main/java/com/nanuq/app/Discovery.java 50
android/app/src/main/java/com/nanuq/app/Downloads.java 50
android/app/src/main/java/com/nanuq/app/LocalServer.java 51
android/app/src/main/java/com/nanuq/app/MainActivity.java 52
android/app/src/main/java/com/nanuq/app/MediaCenter.java 53
android/app/src/main/java/com/nanuq/app/NativeHttp.java 54
android/app/src/main/java/com/nanuq/app/PlaybackService.java 54
android/app/src/main/java/com/nanuq/app/Prefetch.java 54
android/app/src/main/java/com/nanuq/app/SecureStore.java 55
android/bridge/android-bridge.js 56
android/build-apk.sh 56
android/app/src/main/java/com/nanuq/app/OggCheck.java 56

19 **Desktop** 57
Electron and its packaging.

linux/main.js 57
linux/preload.js 59
linux/prepare-web.js 59
linux/collect.js 59
linux/flatpak/build-flatpak.sh 59

20 **Build** 60
The script that builds everything.

scripts/build.sh 60

PART 1

Architecture

How Nanuq is built: a single web interface, the apps that wrap it and how it talks to Jellyfin, to your other devices and to the party.

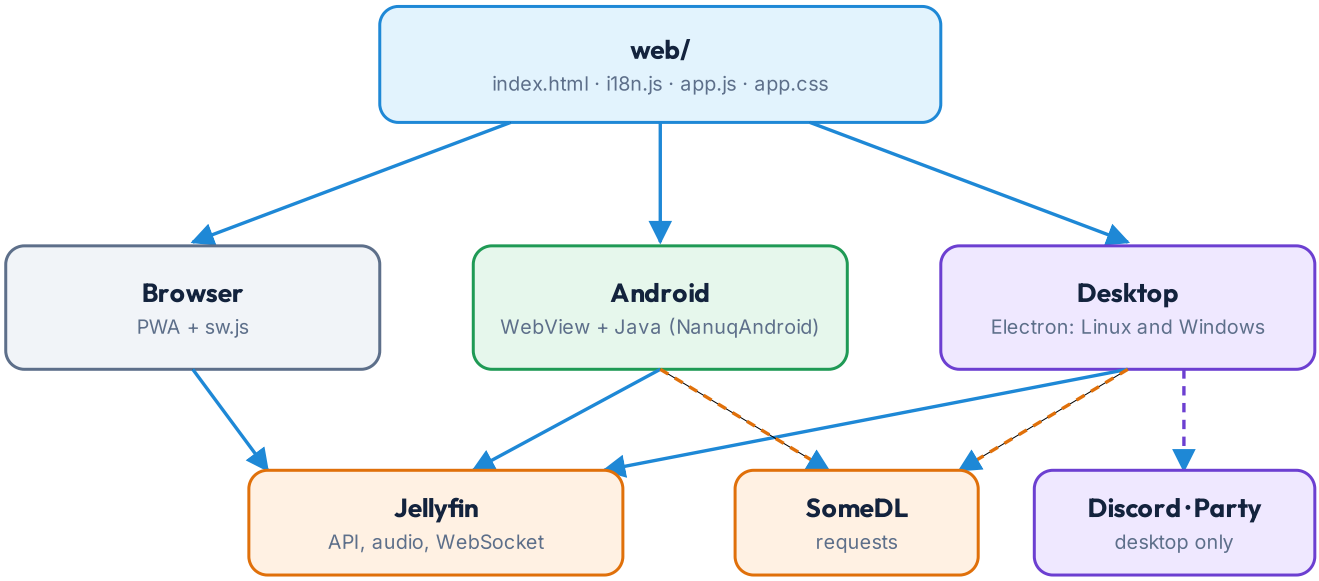


01 Overview

One interface, three wrappers.



All of Nanuq's interface lives in `web/`: four files, with no build step and no dependencies. The Android and desktop apps have no interface of their own: they copy that same folder at build time and add, through a “bridge”, what a web page can't do on its own.



Every platform serves the same page; dashed arrows are what only native code does (SomeDL without CORS, Discord, the party room).

Why this way

- **One change reaches every platform**: the interface is only edited in `web/`; Android and Linux copy it at build time (`syncWeb` in Gradle, `prepare-web.js` in Electron).
- **Secure context without HTTPS**: Android serves the page at `http://localhost` and Electron at `http://127.0.0.1:47863`. That gives `crypto.randomUUID` and storage (the service worker is only used in the browser), while still talking to an `http://` Jellyfin on the local network without mixed-content blocks.
- **No third-party services**: Nanuq only talks to your Jellyfin, your SomeDL and (if you enable it) the Discord app on your computer. The only external requests are the Google Fonts.

Ports

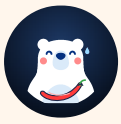
Port	Who	What for
TCP 47863	Desktop (127.0.0.1)	Serves the page and, with the party open, the room's audio (`/fiesta/<token>/…`). Fixed port because storage depends on the origin.
TCP 47864	Desktop (local network)	Party WebSocket (`/fiesta`) and covers for guests. Only with the room open and only private addresses.
UDP 47865	Desktop	Answers “Who is NanuqParty?” so phones find the room (≤ 30 answers/s).
UDP 7359	Jellyfin	Official server discovery (“Who is JellyfinServer?”), from Android and desktop.
TCP random	Android (127.0.0.1)	`LocalServer` : downloads, covers and prefetched songs, with a token in the path.
8096 / 8920	Jellyfin	Ports assumed if you type an IP or short name without a port.
5000	SomeDL	Port assumed for SomeDL without a port.

02 The repository

What lives in each folder.



Path	Contents
`web/index.html`	Structure, strict CSP and SVG `<defs>`: `#i-*` icons and the vector bear (`#djb-*`).
`web/i18n.js`	Languages: `LANGS`, `LANG`, `LOC`, the `EN` dictionary (key = Spanish text) and `tx()`. Loaded before app.js.
`web/app.js`	All the logic (6,062 lines). `const VERSION` is the source of truth for the version.
`web/app.css`	All styling, with per-theme variables (18 themes).
`web/sw.js`	Service worker, browser only: caches the app (network first for code), never music or API. Android and desktop don't register it.
`android/`	WebView + Java (`com.nanuq.app`), the `bridge/android-bridge.js` bridge and `build-apk.sh`.
`linux/`	Electron: `main.js`, `preload.js`, `prepare-web.js`, `collect.js` and `flatpak/`. It also builds the Windows installer.
`scripts/build.sh`	Bumps the version and builds everything into `dist/<version>/`.
`docs/`	README images and these manuals (`docs/manuals/src`: `node docs/manuals/src/build.js`).



Heads-up

`dist/`, `keystore.properties` and `*.jks` files never go into the repository. The APK signing key lives outside (`~/.android/nanuq-release.jks`): without it, updates can't be published.

03 The page

How app.js starts, navigates and reacts.



Startup

1. On load, `i18n.js` picks the language and `txStatic()` translates the fixed HTML texts.
2. `SEC` detects the native encrypted store and `secMigrate()` moves there whatever older versions left in plain text.
3. The boot block draws the sidebar and tabs, applies the theme and starts downloads (`DL.init`).
4. `Vault.init()` retries for a few seconds if the Keystore or keyring isn't ready yet; then `Vault.load()` reads the session.
5. With a session: `startApp(new Jellyfin(sess))` restores the queue and position (`QPOS`), restores the DJ and opens Home. Without one: `showLogin()`.

Navigation

`S.stack` is the screen stack. `go(name, params, opts)` pushes a screen (the `TOP` ones clear the stack) and calls `render()`, which runs `V[name](box, params)`. Each view is an async function that fills its box; if it leaves a `_cleanup`, it's called on exit. `back()` first closes whatever is on top (menu, lyrics editor, player) and then goes back. With the party open (`party-host` / `in-party`) only the party screen exists.

Events

The CSP forbids inline scripts, so there's no `onclick=`: every button has `data-act="…"` and a single global click handler decides what to do (“Global actions” section). Images that must disappear when they fail carry `data-rm`.

Languages

Every visible text is written in Spanish inside `tx('…')`; `i18n.js` holds the translation keyed by the Spanish text. With values: `tx('… {0} …', [x])`. Switching language saves the position and reloads the page, so texts built at load time are translated too.

Sections of app.js

Section	From line
Base: helpers and storage	1
Server addresses	122

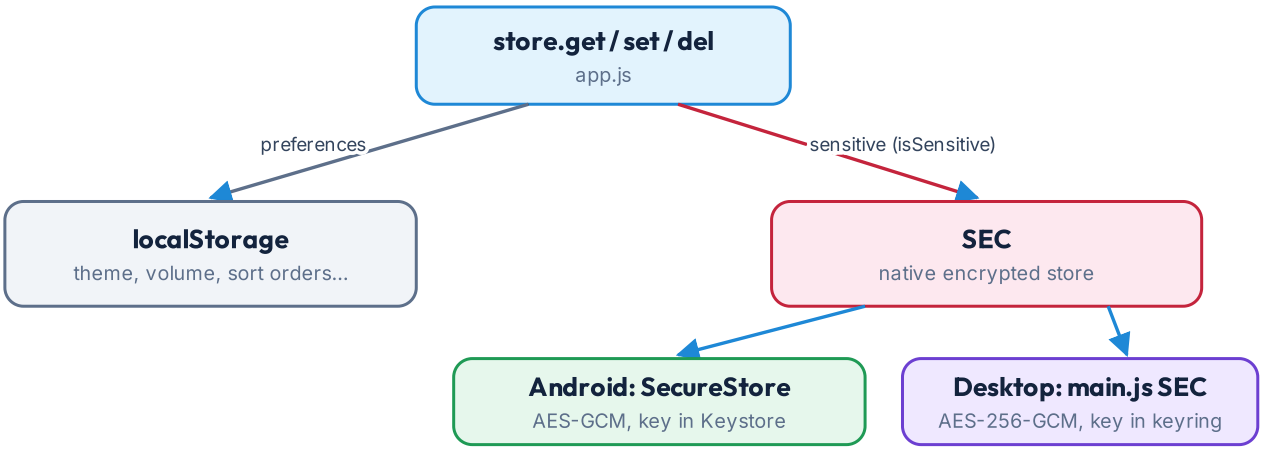
Section	From line
Jellyfin API	177
Audio: waveforms and synthesizer	494
Demo library	555
Settings and themes	753
Downloads (Android)	804
State and routing	957
Albums from tags	1007
Lyrics search	1112
Rendering helpers	1210
Views	1240
DJ Nanuq	1307
Your year and Your week (video)	2263
Song requests (SomeDL)	2979
Player	3270
Lyrics	3823
Queue	4169
Toasts, menus and dialogs	4240
My devices	4251
Party (watch party)	4949
Global actions	5693
Login	5853
Boot	6033

04 Data and storage

Anything about you is encrypted.



`store` is the only way to storage. Interface preferences (theme, volume, sort orders) go to `localStorage` with the `nanuq.` prefix; anything that says something about you goes to the platform's encrypted store. Since preferences aren't encrypted, they're validated when read (right after `Settings`): an altered or outdated value goes back to the default.



In the browser there is no secure store: everything stays in localStorage (Settings says so).

Key	What it holds
session	Jellyfin server, user and token.
queue · queuePos	Your queue (up to 3000) and the second you were at (queuePos isn't sensitive: just a random id and seconds).
libIdx · lyrIdx	Indexes of your library (albums from tags) and of your lyrics.
dj · djHist	Current DJ session and history of what it played.
plays · mp3ids	Plays per song on this device and songs that go as MP3.
somedl · somedlReqs · somedlLast	SomeDL server and your requests.
servers · lastServer · device · party	Recent servers, device id and the current party (phone).
lrc.<id>	Lyrics saved only on this device.

Store states

`SEC.kind` can be the store's name (works), `locked` (encrypted data that won't open right now: retried), `lost` (the key is gone: “Start over” is offered, which sets the data aside without deleting it) or `null` (no store). While it isn't open, new data lives only in memory (`SEC.mem`): it's **never** written in plain text.

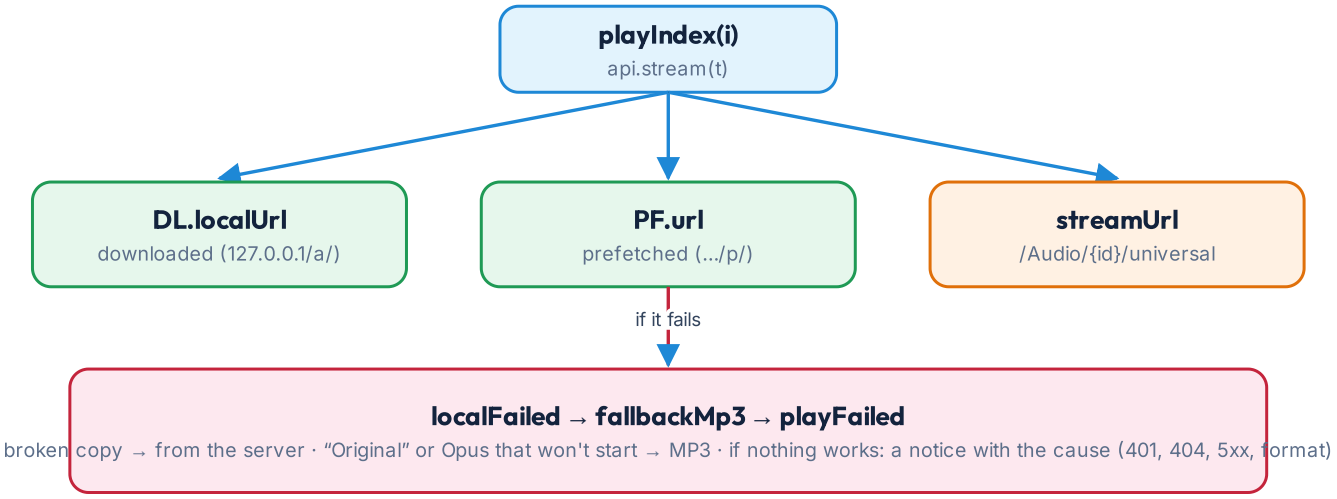
05

Playback

Where each song comes from and what happens when something fails.



`P` holds the queue, the index and two `<audio>` (`P.els`; `P.audio` is always the current song's). `playIndex(i)` picks the source with `api.stream()`: first the downloaded copy, then the prefetched one and, if neither exists, Jellyfin streaming.



On Android, the first two sources are served by the phone itself; on Android and desktop the page never loads the whole audio into memory.

Details that matter

- **Seeking in live transcodes**: if Jellyfin transcodes, the file can't be seeked; `seekTo` requests the audio again with `StartTimeTicks` and remembers the offset (`P.off`). While dragging, the request goes out on release.
- **Stalls**: `armStallWatch` warns if it doesn't start within 10 s; `stallCheck` resumes from the same second if the network dropped mid-song; `netBack` does it when the network returns.
- **Formats**: `streamUrl` lets FLAC, Opus, AAC, MP3, WAV and OGG play as-is (in "Original" or under the bitrate); if it must convert, it asks for Opus in Ogg (`Settings.codec`, if the player opens it: `OPUS_OK`) or MP3. Anything that failed once goes as MP3 (`MP3.ids`). Downloads use the same format.
- **Fades**: `pauseSoft` / `playSoft` with `ramp()`, linear in decibels (−50 dB = silence), from 0.5 to 5 s (`Settings.fadeMs`). While fading out it already counts as paused (`FADE.out`) and ▶ reverses it. The restored volume is always yours (`userVol`). Dragging the progress bar uses 30–40 ms ramps only against clicks (`P.scrub`).
- **DJ crossfade**: `xfCheck` on `timeupdate`; 3–5 s before the end (`Settings.xfade`), `xfStart` swaps the two `<audio>` and crosses them with constant-power (sine/cosine) curves. Events from the outgoing one are ignored (`onAudio`); `xfKill` stops it on pause or song change.

- **Foreign pause**: if another app took the audio output (`P.extPause`), resuming reloads the song so it doesn't stay silent.
- **System controls**: `navigator.mediaSession`; on Android the native bridge replaces it. On desktop a silent audio (`rcKeep` / `sessionHold`) keeps the controls while loading or while you control another device.
- **Favorites**: Jellyfin 12 overwrites the favorite of the playing song with every playback report; `FAVHOLD` saves it again after each report.

06

Library and Jellyfin

The API Nanuq uses and what it builds on its own.



Jellyfin route	What for
/System/Info/Public · /Users/AuthenticateByName	Recognize the server and log in.
/Items · /Artists/AlbumArtists · /MusicGenres · /Playlists	Library lists (with several fallback routes depending on the version).
/Audio/{id}/universal	The audio (streaming, downloads, prefetch, party).
/Audio/{id}/Lyrics · .../RemoteSearch/Lyrics	Read, upload and search lyrics (10.9+).
/UserFavoriteItems/{id}	Favorites (and the pre-10.9 route as fallback).
/Items/{id}/InstantMix	Song radio.
/Sessions · /Sessions/{id}/Playing · /socket	My devices: sessions, commands and live events.
/Sessions/Playing(/Progress|/Stopped)	Playback reports (what the server sees).
/DisplayPreferences/nanuq-dj	DJ state shared between your devices.
/user_usage_stats/...	Playback Reporting plugin for Your week / Your year (if present).

Albums from tags (LIB)

Jellyfin builds albums by folder; Nanuq builds them from each song's “Album” + “Album artist” tags (`albumKeyOf`), joins “(Disc 2)” with its album and removes duplicates. The index of all your songs is fetched 1000 at a time (three pages at once), stored encrypted and refreshed every 10 minutes. These album ids start with `m` (`albumGid`).

Lyrics search (LYRX)

Jellyfin doesn't search inside lyrics: Nanuq indexes the songs with `HasLyrics` (3 at a time, up to 8000) plus local lyrics, normalized without accents or punctuation (`lyrNorm`). Searches need 4 characters or more.

07

DJ Nanuq

Rule-based rounds, no AI.



`DJ.next()` shuffles round types by weight, avoids repeating recent ones and tries until it finds one that **explores** (songs that didn't play in recent weeks). Each song carries its round in the queue id: `dj—<session>-<round>-<random>`.

Round	Weight	What it plays
genre	6	A genre from your library (≥ 3 songs in a 400-song sample), not repeating recent ones.
decade	3.5	A real decade from your library; past ones weigh ×1.5.
discover	3	Never played, first in your favorite genres.
forgotten	3	More than 45 days without playing.
mood	1	By time of day (calm morning, energetic afternoon…).
fresh · fav · recent · top	.8 · .6 · .15 · .15	Newly added, favorites, recent days and most played (with cooldown).

`DJ.rank()` decides how “worn out” a song is: anything from the last 3 days or from recent DJ sessions barely gets in; forgotten songs go first. When music moves to another device, `DJSHARE` leaves the state in your Jellyfin and the other one adopts it (`DJ.adopt`); whatever is read is cleaned with `djClean`. While the first round is built, `DJL` shows Nanuq at the turntable with lines (`DJ_LOAD_SAYS`) until it plays.

08 My devices

One Nanuq plays and the others control it, through Jellyfin.





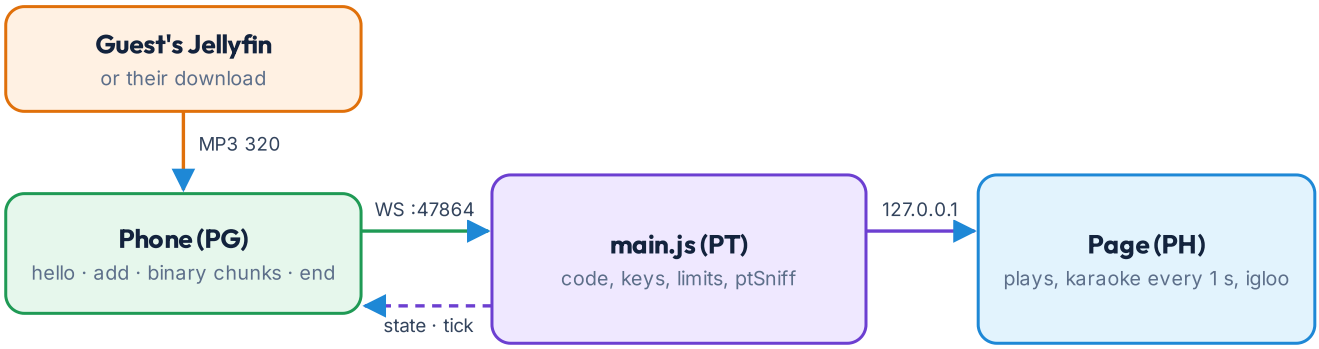
No port is opened on the devices: everything goes through your Jellyfin, with your session.

- **Who shows up**: sessions with client “Nanuq”, of your same user, active within 15 minutes by the *server's* clock (skewed clocks don't matter).
- **Same network**: same /24 in IPv4 or /64 in IPv6 (or both private), using the address the server sees.
- **Accepted commands**: `Play` and `GeneralCommand` only if your user sent them (data Jellyfin fills in); `Playstate` is filtered by Jellyfin permissions.
- **While controlling**, your local queue is set aside (`RC.localQ`) and comes back when you disconnect; the position is estimated from the last report (`RC.anchor`).
- On Android, with the screen locked, the native app calls `__nanuqTick` every 20 s to keep the WebSocket alive.

09 The party

Phones sending songs to a computer over the local network.





UDP :47865 "Who is NanuqParty?" → room name and port

The phone fetches the song from ITS Jellyfin and hands it to the computer in 256 KB chunks, without holding it whole in memory.

Message	Direction	What it is
hello {code|key, name, theme}	phone → PC	Join with the 6-digit code or come back with its key.
welcome {you, key, room, chunk}	PC → phone	Joined: its key to come back without the code (2 h).
add · [id + bytes] · end · remove	phone → PC	Send a song (data, lyrics, cover) and its bytes.
addOk · addErr · ready	PC → phone	Answer to the request and notice that it's ready.
state · tick	PC → phones	List, guests and what's playing; second and karaoke line.
bye {why} · leave · ping	both	Leave, be removed or say it's still there.

Limits and defenses

`PT_MAX` in `main.js`: `{ guests: 12, conns: 24, file: 80e6, disk: 800e6, list: 60, perGuest: 10, frame: 1 << 20, text: 450e3, lyrics: 200e3, cover: 150e3 }`. Private addresses only (`ptLocal`), ≤ 4 not-yet-identified connections per address, 5 wrong codes per minute per address and, after 30 failures overall, a new code and one minute closed. JSON is read with `jparse` and texts with `ptStr`; a message that breaks something only drops that connection.

Songs on disk, encrypted

Songs go to a folder on disk (`ptBase()`: `~/.cache/nanuq/nanuq-fiesta-*` on Linux; in the Flatpak, inside `~/.var/app/com.nanuq.app/cache`), not to the temp folder, which is RAM there. A song is refused if less

than 1 GB is free. Each song is stored **encrypted and authenticated** with AES-256-GCM in 64 KB blocks (`PT_CH`): the key is random and lives only in the room's memory (`PT.fKey`), and the nonce is 8 random bytes per song plus the block number (`ptNonce`). `ptSeal` encrypts on arrival; `ptServe` checks the exact size (`ptEncSize`) and `ptOpenRange` decrypts only the blocks of the requested range. If another app changes a byte, truncates the file or swaps blocks, that block isn't served and the song leaves the list.

10 The Android app

WebView + native Java.

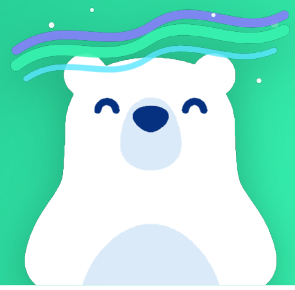


Class	Responsibility
MainActivity	WebView (assets at `http://localhost`), the `NanuqAndroid` bridge, 20 s heartbeat, recreate the WebView if it dies, external links http(s) only.
MediaCenter · PlaybackService	MediaSession and notification; foreground service with CPU and Wi-Fi locks while playing; released after 10 min paused.
Downloads	One-at-a-time downloads, Wi-Fi only by default, limit with replacement of automatic ones; encrypted index; Opus only if its Ogg pages are complete (`OggCheck`).
Prefetch	What's next in the queue, in the cache (≤ 400 MB per file, 1 GB total, 300 MB free).
LocalServer	HTTP on 127.0.0.1 with a token: `a/` audio, `c/` cover, `p/` prefetch; Range; 10 s per request, ≤ 100 headers, 32 threads.
SecureStore	AES-GCM store with a key in Android Keystore; locked / lost states.
NativeHttp · Discovery	HTTP without CORS only for SomeDL routes; UDP discovery for Jellyfin and parties.

The `android-bridge.js` bridge is injected into `index.html` at build time and replaces `navigator.mediaSession` with one that talks to `MediaCenter`. Only Jellyfin-shaped URLs are downloaded (`Downloads.okUrl`) and redirects only within the same server (`openSameHost`).

11 The desktop app

Electron for Linux and Windows.



IPC channel	What it does
nanuq:discover	Finds Jellyfin servers over UDP.
nanuq:http	HTTP without CORS, SomeDL routes only, no redirects, ≤ 4 MB.
nanuq:sec (sync) · nanuq:secreset	Encrypted store: random AES-256 key saved with `safeStorage` (keyring / DPAPI).
nanuq:discord	Rich Presence over the `discord-ipc-N` socket (only sockets you own), with a minimal retry and status for Settings.
nanuq:party · nanuq:party-ev	Open/close the room and run it (jump, order, played, now, tick, lyrics).

- Every IPC channel checks `fromApp(e)`: only the page from our origin can call it.
- Window with `contextIsolation`, `sandbox` and no Node; permissions only for notifications, fullscreen and clipboard.
- Electron fuses: no `runAsNode` or `NODE_OPTIONS`, asar integrity and load only from the asar.
- **Code integrity:** no HTTP disk cache (`disable-http-cache`; the old `Cache` folder is deleted at startup) and, before the page opens, the service worker, Cache Storage and V8 code cache are cleared (also when reloading with F5). `window.json` only accepts sensible numbers.
- Local server: GET/HEAD only, `Host` 127.0.0.1/localhost (against DNS rebinding), paths ≤ 512 and no null bytes.

12 Security

Nothing from outside is trusted.



- **CSP** without inline scripts, without `eval`, iframes or `<base>`; only `style` attributes.
- **Outside JSON** through `jparse` (no `toString` / `valueOf` / `__proto__`) and texts through `str()`: never `String(x)` on foreign data.
- **HTML**: every value goes through `esc()`; ids through `sid()` (letters, digits and dashes only) before going into a path.
- **Lyrics** stripped of control characters and tags (`cleanLyricsText`), ≤ 200,000 characters.
- **Native bridges** with route allowlists and size limits; no following redirects to another server.
- **Personal data** encrypted on disk; Android backup disabled (`allowBackup=false` and extraction rules).
- **Integrity on disk**: nothing stored is used without checking. What's encrypted (personal data, downloads index, party songs) uses AES-GCM, which detects any change; unencrypted preferences are validated when read; on Android everything lives in the app's private storage and an Opus download is only kept if all its Ogg pages arrive complete with their CRC (`OggCheck.valid`; conversions send no size); the server's audio type is only accepted as `audio/…` (`OggCheck.mime`).



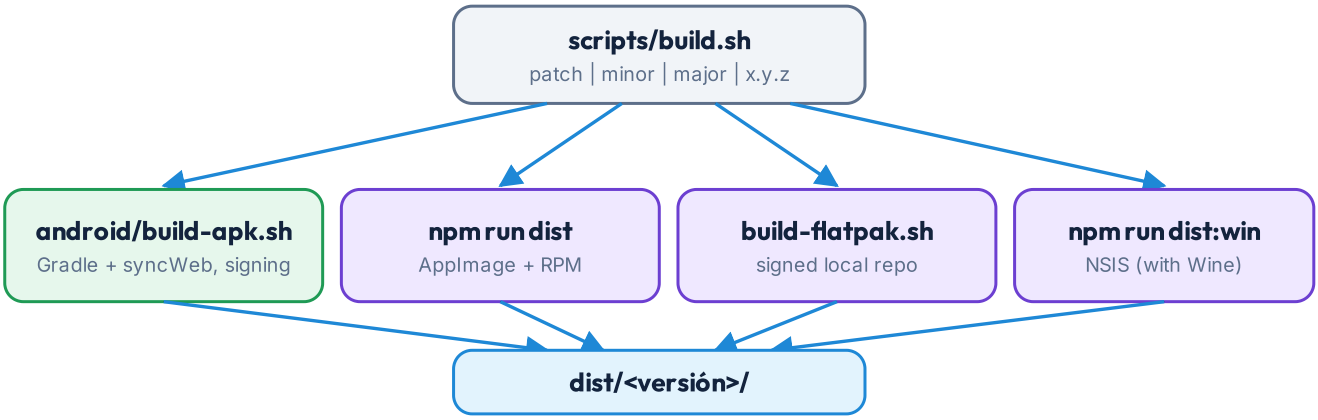
Nanuq's tip

The full detail (what is protected and known limits) is in `SECURITY.md`.

13 Building and releasing

One command for every platform.





The version lives in `web/app.js` (VERSION) and `web/sw.js` (CACHE); the APK, package.json and Flatpak take it from there.

- **Versioning**: new features bump the minor number; fixes, the last one. The APK `versionCode` is M·10000 + m·100 + p.
- **Android** needs JDK 21, the SDK and `android/keystore.properties`. Always signed with the same key.
- **Linux**: the RPM uses the system `fpm` (`USE_SYSTEM_FPM=true`). The Flatpak is built from `linux-unpacked`.
- **Checks**: `node --check web/app.js` and, for UI, Chromium in demo mode.

14 Conventions and pitfalls

What to know before changing anything.



Rule	Why
UI changes only in `web/`	Android and Linux copy it at build time.
Every visible text in `tx('…')` + its `EN` translation	Otherwise it shows in Spanish in English mode. One full sentence per `tx`, no gluing fragments.
No `onclick=`, `<script>` or `<style>` from JS	The CSP blocks them silently.
Unique ids across the whole app	A duplicated id hid the Search box (1.30.2).
Unique `view-transition-name`	If repeated, the transition aborts with no visible error.
Endless animations only in bursts (`nqBurst`)	At 120 Hz each one costs ≈ 1 phone core.
Respect `prefers-reduced-motion`	Accessibility.
After writing to Jellyfin, show the local copy and confirm later	An immediate read may return stale data (lyrics, playlists, favorites).
`.ig-crowd` without `z-index`	It would create its own layer and everyone would end up behind the DJ.

15 License

GPLv3 or later, file by file.



Nanuq is free software under the **GNU General Public License, version 3 or (at the recipient's option) any later version**: `GPL-3.0-or-later`. The full text is in `LICENSE`, at the root of the repository.

What it means for whoever modifies it

- You can use, study, copy and modify the code for any purpose.
- If you **distribute** Nanuq or a modified version (an APK, a package, the PWA on a server others download from), it must go under the same license and with its source code (or an offer to get it), keeping the copyright and license notices.
- A modified version must say it is one. For Android, use another `applicationId` and your own signing key: only the author signs the official APK (see “Building and releasing”).
- It comes with **no warranty**.

How the code is marked

Every source file starts with two lines in the **SPDX** format (the one used by license tools and the REUSE specification):

```
SPDX-FileCopyrightText: 2026 Fleyrer y colaboradores de Nanuq
SPDX-License-Identifier: GPL-3.0-or-later
```

They go as a comment in each file's language (`//` , `/* */` , `<!-- -->` or `#`), after the `#!` in scripts and after the `<!DOCTYPE>` in HTML. When you create a new file, add the same two lines.

Where else it shows up

Place	How
`LICENSE`	Official GPLv3 text (GitHub detects it and shows it on the repository).
README.md · README.en.md	Line at the top, “License” section and a link in the menu.
App settings	“Nanuq x.y.z” row: the license and a link to `LICENSE` (opens in the browser).
linux/package.json	`"license": "GPL-3.0-or-later"`
Flatpak (metainfo)	`<project_license>GPL-3.0-or-later</project_license>`
These manuals	Cover and this chapter.

What isn't Nanuq's

Component	License	Where
Electron	MIT	Inside the desktop apps (Linux and Windows).
Inter · Outfit	SIL Open Font License	Fonts, loaded from Google Fonts (not in the repository).
Gradle wrapper	Apache 2.0	`android/gradlew`
Jellyfin	GPL-2.0	The server: a separate project Nanuq talks to through its API. Nanuq is not affiliated with it.

PART 2

Deep level: every function

All 860 functions and methods in the code, file by file, with the line where they start and what they do.



16 web/app.js

598 functions, in file order.



web/app.js 6,062 lines

Base: helpers and storage from line 1 · 25

Function	Line	What it does
$	5	Shortcut for document.querySelector (optionally inside another element).
$$	6	querySelectorAll shortcut that returns an array.
jparse	10	Parses outside JSON without toString/valueOf/__proto__.
str	11	Converts any value to text without ever throwing.
esc	12	Escapes text to put it into HTML (& < > " ').
sid	16	Cleans a server id: only letters, numbers and dashes.
num	17	Converts to a number; 0 if it isn't finite.
numOrNull	18	Converts to a number or returns null if empty or invalid.
ic	30	HTML for an SVG icon from the <defs> (#i-name).
fmt	31	Formats seconds as m:ss or h:mm:ss.
isSensitive	38	Does this stored key hold personal data (goes to the encrypted store)?
SEC.ok	46	Is the native encrypted store open and usable?
SEC.shut	47	Is the store locked or lost (nothing is written in plain text)?
store.get	55	Reads a stored value (encrypted if sensitive, otherwise localStorage) and migrates old plain copies.
store.set	67	Saves a value; sensitive data goes to the encrypted store or, if closed, only to memory.
store.del	78	Deletes a stored value from the store and localStorage.
secMigrate	81	Moves into the encrypted store what older versions left in plain text.
Vault.kind	95	Name of the encrypted store in use (or null).
Vault.init	98	Cold start: retries opening the store for a few seconds before giving up.
Vault.save	101	Saves the Jellyfin session.
Vault.load	102	Reads the saved session (or flags that the store is locked).

Function	Line	What it does
`Vault.reset`	109	“Start over”: sets unreadable data aside and creates a new store.
`uid`	117	Random id (UUID without dashes).
`deviceId`	118	Persistent id of this device for Jellyfin.
`deviceName`	119	System name (Android, Linux, Windows…) announced to Jellyfin.

Server addresses from line 122 · 1

Function	Line	What it does
`parseServer`	124	Parses what you typed as a server (IPv4, IPv6, name, port and path) and builds the base URL.

Jellyfin API from line 177 · 58

Function	Line	What it does
`Jellyfin.constructor`	180	Creates the Jellyfin client from the saved session.
`Jellyfin.authHeader`	181	Builds the MediaBrowser authorization header with device, version and token.
`Jellyfin.req`	185	API request with a timeout and no cache; detects 401 and reads JSON with jparse.
`Jellyfin.alt`	203	Tries several API routes until one doesn't return 404.
`Jellyfin.probe`	204	Checks that an address is a Jellyfin (/System/Info/Public).
`Jellyfin.login`	212	Logs in with username and password and stores the token.
`Jellyfin.cfg`	216	Session data to save.
`Jellyfin.map`	217	Converts a Jellyfin item to Nanuq's format, cleaning ids, numbers and texts.
`Jellyfin.items`	236	Generic /Items query with the fields the app uses.
`Jellyfin.albums`	240	Paginated, sorted Jellyfin albums.
`Jellyfin.item`	244	One item by id (with a fallback route for old servers).
`Jellyfin.album`	245	A Jellyfin album and its tracks.
`Jellyfin.artists`	249	Album artists, paginated.
`Jellyfin.artistFilters`	254	Possible filters to find an artist's items (by several ids or by name).
`Jellyfin.firstNonEmpty`	260	Tries filters in order and returns the first non-empty result.
`Jellyfin.artist`	264	An artist and their albums (built from their songs if needed).
`Jellyfin.artistTracks`	279	All of an artist's songs.
`Jellyfin.genres`	287	Music genres trying several routes; otherwise derived from albums.
`Jellyfin.genreParam`	306	Query parameters for one or more genres (by id or by name).

Function	Line	What it does
`Jellyfin.genre`	311	A genre's albums.
`Jellyfin.genreTracks`	318	A genre's songs (from the songs' tags or their albums).
`Jellyfin.playlists`	327	Your audio playlists.
`Jellyfin.playlist`	328	A playlist and its songs.
`Jellyfin.views`	332	The user's music libraries (root of Folders).
`Jellyfin.folder`	336	A folder's contents.
`Jellyfin.search`	337	Search for artists, albums and songs.
`Jellyfin.img`	344	Cover URL (local if it's downloaded on the phone).
`Jellyfin.streamUrl`	351	Universal streaming URL: formats that play as-is and conversion to Opus or MP3.
`Jellyfin.stream`	357	URL to play: download, ahead-of-time cache or streaming.
`Jellyfin.dlUrl`	360	URL to download a song to the phone (Opus or MP3, or the file as-is).
`Jellyfin.peaks`	367	Downloads a light copy and computes the real waveform.
`Jellyfin.tracks`	372	Paginated, sorted library songs.
`Jellyfin.lyrics`	376	A song's lyrics (Jellyfin 10.9+), normalized.
`Jellyfin.mix`	381	Songs for a mix (order, filters, genres, years).
`Jellyfin.uploadLyrics`	388	Uploads lyrics (.lrc or .txt) to Jellyfin.
`Jellyfin.searchLyrics`	395	Searches for lyrics in Jellyfin's providers (LrcLib).
`Jellyfin.pickLyrics`	399	Picks one of the found lyrics to save it.
`Jellyfin.setFavorite`	401	Marks or unmarks a favorite.
`Jellyfin.inFavorites`	407	Is it in Jellyfin's favorites section?
`Jellyfin.isFavorite`	408	Is this song a favorite according to its user data?
`Jellyfin.favPlaylist`	415	Finds the “Favorited Songs Playlist” plugin's playlist.
`Jellyfin.favorites`	422	Your favorites (IsFavorite filter), paginated and sorted.
`Jellyfin.setFavoriteLegacy`	440	Old route to save favorites (second attempt).
`Jellyfin.instantMix`	442	Jellyfin instant mix from a song (radio).
`Jellyfin.addToPlaylist`	447	Adds songs to a playlist.
`Jellyfin.refreshLibrary`	449	Asks Jellyfin to rescan the libraries (administrators).
`Jellyfin.savePlaylist`	450	Creates a new playlist with songs.
`Jellyfin.report`	451	Tells Jellyfin a song started, progressed or stopped (and reasserts favorites).
`Jellyfin.logout`	457	Logs out on the server (revokes the token).
`Jellyfin.wsUrl`	460	Jellyfin WebSocket URL.

Function	Line	What it does
`Jellyfin.postCapabilities`	461	Announces that this Nanuq accepts remote-control commands.
`Jellyfin.sessions`	463	Active sessions (for My devices).
`Jellyfin.sessionCmd`	464	Sends a playback command to another session.
`Jellyfin.sessionPlay`	465	Sends a queue to play on another session.
`Jellyfin.sessionGeneral`	466	Sends a general command (repeat, shuffle) to another session.
`Jellyfin.itemsByIds`	467	Requests several items by id, 100 at a time.
`netError`	473	Clear message when the server can't be reached.
`normLyrics`	482	Normalizes Jellyfin lyrics (synced or plain).

Audio: waveforms and synthesizer from line 494 · 6

Function	Line	What it does
`actx`	497	Shared AudioContext (created on first use).
`decode`	498	Decodes audio into an AudioBuffer.
`peaksFromBuffer`	499	Computes waveform peaks from an AudioBuffer.
`rng`	506	Seeded pseudo-random generator.
`wav`	507	Converts an AudioBuffer into a WAV file.
`synth`	514	Synthesizes a test song for demo mode.

Demo library from line 555 · 43

Function	Line	What it does
`Demo.constructor`	558	Creates the demo library (artists, albums, songs and playlists).
`Demo.cfg`	593	Demo-mode session.
`Demo.wait`	594	Simulates network latency.
`Demo.albums`	595	Demo version of Jellyfin.albums.
`Demo.album`	600	Demo version of Jellyfin.album.
`Demo.artists`	601	Demo version of Jellyfin.artists.
`Demo.artistBy`	603	Finds a demo artist by id or name.
`Demo.artist`	604	Demo version of Jellyfin.artist.
`Demo.artistTracks`	605	Demo version of Jellyfin.artistTracks.
`Demo.genres`	606	Demo version of Jellyfin.genres.
`Demo.genreTracks`	607	Demo version of Jellyfin.genreTracks.
`Demo.genre`	608	Demo version of Jellyfin.genre.
`Demo.playlists`	609	Demo version of Jellyfin.playlists.

Function	Line	What it does
`Demo.playlist`	610	Demo version of Jellyfin.playlist.
`Demo.views`	611	Demo version of Jellyfin.views.
`Demo.folder`	612	Demo version of Jellyfin.folder.
`Demo.search`	617	Demo version of Jellyfin.search.
`Demo.img`	621	Draws a generated SVG cover for demo mode.
`Demo.buf`	635	Synthesized audio for a demo song (cached).
`Demo.stream`	636	Blob URL of a demo song's WAV.
`Demo.peaks`	637	Waveform of a demo song.
`Demo.tracks`	638	Demo version of Jellyfin.tracks.
`Demo.mix`	644	Demo version of Jellyfin.mix.
`Demo.lyrics`	654	Made-up synced lyrics for a demo song.
`Demo.setFavorite`	666	Demo version of Jellyfin.setFavorite.
`Demo.isFavorite`	667	Demo version of Jellyfin.isFavorite.
`Demo.inFavorites`	668	Demo version of Jellyfin.inFavorites.
`Demo.favorites`	669	Demo version of Jellyfin.favorites.
`Demo.instantMix`	670	Demo version of Jellyfin.instantMix.
`Demo.addToPlaylist`	671	Demo version of Jellyfin.addToPlaylist.
`Demo.savePlaylist`	672	Demo version of Jellyfin.savePlaylist.
`Demo.report`	673	Does nothing (demo doesn't report plays).
`Demo.logout`	674	Does nothing (no server).
`logoInner`	679	HTML of the logo bear with the theme's accessory.
`bearHTML`	695	HTML of the animatable Nanuq (logo + expressive face).
`bearState`	697	Current bear state (dancing, happy, sad) for new bears.
`nqBurst`	703	Turns on an animation burst for a few seconds (battery saving).
`nqBurstEnd`	711	Ends the burst: animations pause and bears stay happy.
`bearDance`	716	Makes every Nanuq dance for a while.
`bearInit`	721	Puts Nanuq in the sidebar logos.
`bearMood`	726	Changes the bears' mood when music plays or pauses.
`showSplash`	737	Welcome screen with the theme's Nanuq.
`refreshBear`	748	Changes the bears' accessory when the theme changes.

Settings and themes from line 753 · 2

Function	Line	What it does
OPUS_OK	756	Does the player open Opus? Decides whether it is offered and used by default.
Settings.set	785	Changes a setting and saves it.

Downloads (Android) from line 804 · 29

Function	Line	What it does
DL.init	810	Loads the phone's downloads from the native app.
DL.indexCovers	818	Index of downloaded covers by image id.
DL.coverUrl	822	Local URL of a downloaded cover.
DL.backfillCovers	824	Fills in covers and details of old or recovered downloads.
DL.config	831	Sends Android the maximum songs and whether Wi-Fi only.
DL.usable	832	Are downloads available on this device (Android with Jellyfin)?
DL.get	833	A song's download, if any.
DL.localUrl	834	Local URL of a downloaded, ready song.
DL.isLocal	835	Is this URL from the phone's local server?
DL.add	837	Requests downloading a song (manual or automatic).
DL.remove	842	Removes a song from downloads.
DL.removeAll	843	Deletes all downloads (or only automatic ones).
DL.list	844	List of downloads.
DL.bytes	845	Total space used by downloads.
DL.net	846	Current network type according to Android (wifi, cellular, none).
__nanuqDl	848	Receives a download's changes from Android and updates the view.
Plays.count	863	Times a song played on this phone.
Plays.add	864	Counts a play and auto-saves the song at 3.
PF.usable	881	Can it save ahead of time (Android with Jellyfin)?
PF.isUrl	882	Is this URL from the ahead-of-time cache?
PF.url	883	Local URL of a song already saved ahead of time (or null).
PF.soon	884	Schedules an update of the ahead-of-time window.
PF.sync	885	Computes which queue songs to save (N minutes) and asks Android.
fmtMB	900	Formats bytes as MB or GB according to the language.
dlStatus	901	Text of a download's state.

Function	Line	What it does
dlStatusShort	912	Short download state for the player button.
dlSummary	918	Downloads summary (count, space, pending).
themeSummary	942	HTML of the chosen theme in Settings.
applyTheme	946	Applies a theme (colors, system bar, bears).

State and routing from line 957 · 8

Function	Line	What it does
regList	963	Registers a song list so buttons can find it.
cur	972	Current screen in the navigation stack.
go	973	Navigates to a screen (root or on top of the current one).
back	982	Goes back: closes dialogs, the player or the current screen.
cacheDrop	992	Forgets cached views starting with a prefix.
refreshIf	994	Redraws the current screen if it meets a condition.
playlistsChanged	997	Forgets and re-requests a playlist that changed.
cached	1004	View cache with a cap (oldest go first).

Albums from tags from line 1007 · 15

Function	Line	What it does
albumNorm	1013	Normalizes a name to compare albums.
albumKeyOf	1015	Album key of a song from its tags (artist + album + disc).
albumGid	1022	Stable id of a tag-built album.
LIB.who	1025	Identifies the index's server and user.
LIB.index	1027	Tag-based album index (from the encrypted cache or the server).
LIB.peek	1040	The index if it's ready, without waiting.
LIB.reset	1041	Marks the index as stale so it's rebuilt.
LIB.refresh	1042	Rebuilds the index with every song.
LIB.fetchAll	1048	Downloads every song in the library (3 pages at a time).
LIB.pack	1058	Compact version of a song to save the index.
LIB.unpack	1059	Reads a compact song from the saved index, cleaning it.
LIB.save	1064	Saves the encrypted index (if it fits).
LIB.build	1069	Groups songs into albums, without duplicates.
LIB.albumsOf	1094	Albums (by tags) of a group of songs.
LIB.sorted	1099	Sorted album list.

Lyrics search from line 1112 · 10

Function	Line	What it does
lyrNorm	1118	Normalizes lyrics text for searching (no accents or punctuation).
LYRX.load	1121	Loads the saved lyrics index.
LYRX.add	1130	Adds a song's lyrics to the index.
LYRX.save	1131	Saves the encrypted lyrics index (debounced).
LYRX.reset	1139	Deletes the lyrics index.
LYRX.build	1141	Builds or completes the index reading lyrics 3 at a time.
LYRX.search	1167	Finds songs whose lyrics contain a text.
lyrSnippetHTML	1184	Lyrics line with the search highlighted (escaped).
lyricsHitsHTML	1189	HTML list of songs found by lyrics.
lyricsSection	1194	“In the lyrics” section that fills in while indexing.

Rendering helpers from line 1210 · 8

Function	Line	What it does
coverHTML	1212	HTML of a cover with its fallback icon.
head	1217	Screen header (title, buttons, bear, back).
albumCard	1221	An album card.
artistCard	1222	An artist card.
trackRows	1223	Container of song rows.
rowsHTML	1224	Song rows (number, title, duration, menu).
loading	1236	Loading indicator.
errBox	1237	Error box with “Retry”.

Views from line 1240 · 6

Function	Line	What it does
V.albums	1243	Albums screen (by tags, 120 at a time).
V.album	1258	An album's screen.
normTxt	1289	Normalizes text (no accents, lowercase).
mixDef	1290	A mix's definition (query by genres or years).
mixTracks	1304	A mix's songs (cached).
greeting	1305	Greeting by time of day.

DJ Nanuq from line 1307 · 85

Function	Line	What it does
djBearSVG	1313	SVG of DJ Nanuq with headphones, scarf and turntable.

Function	Line	What it does
djMoment	1365	Time of day and mood for the “For right now” round.
pick	1366	Picks one at random.
wpick	1368	Picks at random with weights.
wshuffle	1370	Shuffles giving more chance to heavier items.
djQid	1373	Queue id carrying the DJ session and round.
djTag	1374	Reads session and round from a DJ queue id.
djSame	1375	Are two ids from the same DJ session?
DJ.voiceOk	1378	Is there a system voice in the app's language?
DJ.save	1379	Saves the DJ session and shares it with your devices.
DJ.restore	1384	Restores the DJ session when the app opens.
DJ.sample	1391	Random sample of the library to choose genres and decades.
DJ.heavy	1397	Your 60 most played songs.
DJ.pickGenre	1403	Picks a round's genre (avoiding recent ones).
DJ.pickDecade	1413	Picks a round's decade.
DJ.rank	1425	How worn out a song is (lower = goes first).
DJ.pool	1434	Candidate songs for a round type.
DJ.build	1467	Builds a round of ~5 songs with its introduction.
DJ.next	1494	Picks and builds the next round (weights and cooldown).
DJ.items	1508	Turns a round into queue songs and records it.
DJ.lastKey	1516	Number of the last round.
DJ.start	1517	Starts a DJ session with its loading screen (here or on the device you control).
DJ.startNow	1522	Builds the first round and waits until it plays (behind the loading screen).
DJ.share	1547	Leaves the DJ state in Jellyfin for another device.
DJ.handedOff	1552	The session moved to another device: it turns off here.
DJ.adopt	1554	Continues here a session coming from another device.
DJ.queueFromShare	1559	Rebuilds the DJ queue from the shared state.
DJ.queueFromTags	1561	Rebuilds the DJ queue from the queue ids.
DJ.remoteTag	1563	Is what's playing on the device you control the DJ?
DJ.shown	1564	Should the DJ be shown?
DJ.remoteTrack	1566	Fetches the rounds of the DJ playing on another device.
DJ.stop	1576	Turns the DJ off (the queue stays as is).

Function	Line	What it does
DJ.onTrack	1578	On song change: introduces the new round and adds the next.
DJ.extend	1586	Adds the next round to the end of the queue.
DJ.switchUp	1596	“Change the vibe”: another round instead of what was left.
DJ.announce	1604	Introduces a round (bubble, notice and voice).
DJ.show	1606	Shows the introduction where appropriate.
DJ.speak	1611	Reads the introduction aloud and lowers the music.
DJ.visible	1629	Is the page visible?
DJ.bears	1630	Turns the DJs' dancing on or off.
DJ.say	1634	DJ speech bubble in the player.
DJ.scene	1639	Record scene in the player (sleeve, vinyl, entries and exits).
DJ.paint	1658	Updates everything DJ-related on screen.
DJSHARE.ok	1688	Can the DJ be shared (Jellyfin and sync on)?
DJSHARE.state	1689	DJ state to share (bounded).
DJSHARE.push	1694	Saves the DJ state in /DisplayPreferences.
DJSHARE.pushRaw	1702	Saves an already built state.
DJSHARE.pushSoon	1706	Saves the state in 1.5 s.
DJSHARE.offSoon	1708	Announces the DJ turned off, if the session is still this device's.
DJSHARE.pull	1712	Reads the shared DJ state.
djClean	1718	Cleans the read DJ state (untrusted data).
djLiveSync	1733	Animates the DJ only while it's visible.
djTileHTML	1745	DJ Nanuq card on Home.
DJL.show	1755	Shows the DJ loading screen with rotating lines.
DJL.hide	1765	Hides the loading screen (at least 1 s on screen).
V.dj	1770	DJ Nanuq screen.
nanuqSceneSVG	1794	Nanuq scene for an empty Home cell.
nanuqBear	1810	Nanuq ready to place in a scene.
nanuqWideSVG	1814	Wide Nanuq scene (two cells).
nanuqWideTileHTML	1831	Wide cell with a scene.
nanuqTileHTML	1836	Cell with a scene.
nanuqVisit	1843	A Nanuq from another theme visits a cell.
nanuqVisits	1863	Schedules visits every now and then.
fillGaps	1871	Fills Home's grid gaps with scenes.

Function	Line	What it does
`tileHTML`	1884	A mix tile.
`offlineBanner`	1889	“No connection” notice with access to Downloads.
`V.home`	1894	Home screen.
`V.mix`	1939	A mix's screen.
`V.songs`	1949	Songs screen (paginated).
`V.artists`	1977	Artists screen.
`fmtLong`	1984	Long duration (“3 h 12 min”).
`V.artist`	1986	An artist's screen (songs, albums, appears on).
`V.genres`	2066	Genres screen.
`V.genre`	2071	A genre's screen.
`V.playlists`	2082	Playlists screen.
`V.playlist`	2087	A playlist's screen.
`V.favorites`	2096	Favorites screen (confirmed with Jellyfin).
`V.folders`	2122	Folders screen.
`V.search`	2131	Search screen (includes lyrics search).
`songsBlock`	2161	Songs block as a list or grid.
`V.queue`	2168	Queue tab (phone).
`V.more`	2169	More screen.
`V.settings`	2172	Settings screen.
`V.downloads`	2241	Downloads screen.
`renderDownloadsSoon`	2257	Redraws Downloads shortly.

Your year and Your week (video) from line 2263 · 37

Function	Line	What it does
`History.purge`	2271	Deletes local history from earlier versions.
`ServerStats.available`	2281	Is the Playback Reporting plugin present and readable?
`ServerStats.plays`	2289	Plays between two dates from the plugin (day by day).
`Recap.total`	2345	Total video length.
`Recap.setMode`	2346	Switches between year and week recap.
`Recap.plays`	2351	Plays for the period (plugin, basic data or demo).
`Recap.tally`	2368	Groups plays: songs, artists, genres, hours, days, streak.
`Recap.collect`	2392	Year recap data.
`Recap.collectWeek`	2404	Data for last week's recap (Monday to Sunday).
`Recap.jellyfinYear`	2420	Songs played in a period according to Jellyfin's basic data.

Function	Line	What it does
`Recap.assets`	2433	Prepares the video's images: bears, DJ, covers and palette.
`Recap.loadImg`	2455	Loads an image with limits (8 MB, reduced) for the canvas.
`Recap.sceneAt`	2469	Which scene plays at a given second of the video.
`Recap.frame`	2470	Draws a video frame (with transitions between scenes).
`Recap.transition`	2479	Transition between two scenes (slide, circle, zoom, wipe).
`Recap.bg`	2489	A scene's background: gradient, beat glow and snowflakes.
`Recap.weather`	2506	Draws the music forecast (sun, storm, snow, rainbow...).
`Recap.bear`	2523	Draws a Nanuq hopping to the beat.
`Recap.text`	2530	Text fitted to width, with a shadow.
`Recap.cover`	2540	Cover with rounded corners.
`Recap.confetti`	2546	Confetti shower.
`Recap.scene`	2554	Draws a recap scene.
`Recap.music`	2801	Video music generated with Web Audio (with a drumroll).
`Recap.stop`	2830	Stops playing or recording the video.
`Recap.play`	2831	Plays the video on the canvas and, if asked, records it (MediaRecorder).
`Recap.save`	2859	Saves the video (Android in chunks, desktop as a download).
`mp4Finalize`	2885	Writes the real duration into the fragmented MP4.
`recapCloseAudio`	2926	Closes the video's audio only once.
`ease.out`	2927	Animation curve (out).
`ease.inOut`	2927	Animation curve (in-out).
`ease.back`	2927	Animation curve with overshoot.
`clamp01`	2928	Clamps a value between 0 and 1.
`mixHex`	2929	Mixes two hex colors.
`hexA`	2930	Hex color with transparency.
`recapYearOpen`	2932	Is it the first week of December?
`recapTileHTML`	2933	“Your year with Nanuq” card on Home.
`V.recap`	2936	Your week / Your year screen (view and export).

Song requests (SomeDL) from line 2979 · 26

Function	Line	What it does
`__nanuqHttp`	2985	Receives a native SomeDL request's response (Android).
`reqNative`	2986	Is there a native bridge to talk to SomeDL without CORS?
`reqRawHttp`	2987	Request to SomeDL through the native bridge or fetch.
`parseSomedl`	2997	Parses the SomeDL address (port 5000 by default).

Function	Line	What it does
`reqNetError`	3006	Clear message when SomeDL doesn't respond.
`REQ.save`	3021	Saves the request list.
`REQ.setCfg`	3022	Saves or clears the SomeDL address.
`REQ.pending`	3023	Requests still on the way.
`REQ.visible`	3024	Is the app visible (to check more often)?
`REQ.http`	3025	Request to the SomeDL API with error handling.
`REQ.probe`	3033	Checks that an address is SomeDL.
`REQ.add`	3039	Sends a song request to SomeDL.
`REQ.schedule`	3051	Schedules the next request check.
`REQ.poll`	3052	Checks the requests' state in SomeDL.
`REQ.announce`	3081	Announces (and notifies) which requests finished.
`REQ.retry`	3096	Requests a failed song again.
`sysNotify`	3103	System notification (Android or desktop).
`askNotifyPermission`	3109	Asks for notification permission in the browser.
`reqRowHTML`	3112	A SomeDL request row.
`reqListHTML`	3121	Request list.
`renderReqSoon`	3128	Redraws the request list shortly.
`V.request`	3132	Request a song screen.
`refreshFromJellyfin`	3192	“Refresh from Jellyfin”: forgets caches, rebuilds indexes and refreshes the queue.
`sessionGone`	3215	Confirms whether a 401 really is an expired session.
`sessionExpired`	3226	Expired session: stops everything and asks to log in.
`render`	3234	Draws the current screen with a transition and error handling.

Player from line 3270 · 61

Function	Line	What it does
`P.pause`	3280	Pauses, marking that Nanuq requested it.
`onAudio`	3285	Listens to an event on both <audio> elements but only handles the current song's.
`saveQueue`	3287	Saves the queue and updates the ahead-of-time window.
`QPOS.save`	3292	Saves the song's second to resume it.
`QPOS.restore`	3298	Where to resume the song when the app opens.
`QPOS.clear`	3302	Forgets the saved position.
`setQueue`	3306	Replaces the queue and starts playing.

Function	Line	What it does
addToQueue	3307	Adds songs at the end or next.
ptime	3317	Current second of the song (here or on the other device).
pdur	3318	Song duration.
setSrc	3324	Changes the audio source resetting its state.
seekTo	3328	Seeks to a second (for live conversions it requests from there).
playIndex	3349	Plays song N of the queue.
armStallWatch	3380	Warns if a song takes too long to start.
MP3.add	3394	Remembers a song that must be requested as MP3.
localFailed	3398	The local copy failed: it plays from the server (and is downloaded again).
canFallback	3410	Can this song be requested as MP3 (in “Original” or with Opus)?
fallbackMp3	3413	Requests the song as MP3 if the original or Opus one won't start.
armFallback	3423	Schedules the switch to MP3 if it doesn't start in 6 s.
playFailed	3429	Explains why it couldn't play.
audioDead	3450	Is the audio dead (network error)?
replayHere	3451	Requests the song again from the second it was at.
rampStop	3464	Stops a volume ramp.
ramp	3468	Timer-driven volume ramp (in decibels, or constant-power for the crossfade).
fadeStop	3477	Stops a fade in progress.
fadeTo	3478	Fades the volume to a value.
softMs	3479	Pause/play fade length (0 when off).
userVol	3483	The volume you chose (lowered if the DJ is talking).
pauseSoft	3484	Pauses with a fade.
playSoft	3492	Resumes with a fade, or fades back up if it was fading out (reloads if another app paused).
partyNow	3502	Do the controls handle the party right now?
partyHostOpen	3505	Is a party room open on this computer?
toggle	3506	Play/pause (party, remote control or here).
next	3515	Next song (respects repeat and the end of the queue).
prev	3527	Previous song (or restart if past 3 s).
seekFrac	3528	Seeks to a fraction of the song.
removeAt	3529	Removes a song from the queue.

Function	Line	What it does
clearQueue	3534	Clears the queue.
shuffleQueue	3535	Shuffles the queue (the current one stays first).
setMode	3543	Changes the repeat mode.
xfDrop	3551	Releases the <audio> of the song that left.
xfKill	3553	Stops the outgoing crossfade song (on pause or song change).
xfCheck	3557	Decides whether the DJ crossfade starts (3–5 s before the end).
xfStart	3566	Crosses the ending song with the next one using both <audio> elements.
updateModeBtn	3580	Updates the repeat button.
loadPeaks	3582	Loads a song's waveform.
cssVar	3591	Reads a theme CSS variable (cached).
drawWaves	3592	Draws the progress bars or the waveform.
roundRect	3620	Rounded rectangle on canvas.
updateNow	3622	Updates everything that shows the current song.
updatePlayState	3652	Play/pause icons and system controls state.
DISC.can	3667	Is there a Discord bridge (desktop)?
DISC.update	3668	Schedules the update to Discord.
DISC.send	3669	Sends Discord what's playing (or clears it).
dcStatusPaint	3685	Discord status line in Settings.
rcKeep	3706	Silent audio so the system keeps the controls.
sessionHold	3720	Keeps the system controls while loading or when the queue ends.
rcKeepOff	3726	Stops the silent audio.
stallCheck	3766	Detects audio stuck mid-song and requests it again.
netBack	3775	Network is back: resumes what was cut and reconnects.
paintVol	3818	Draws the volume bar and its icon.

Lyrics from line 3823 · 32

Function	Line	What it does
lrcKey	3826	Key of lyrics saved on the device.
lrcTime	3827	Formats a time as mm:ss.cc (LRC).
parseLRC	3828	Reads an LRC file (timestamps and offset).
toLRC	3842	Builds a clean LRC file.
cleanLyricsText	3853	Cleans typed or pasted lyrics (control chars, HTML, size).

Function	Line	What it does
lyricsFromText	3862	Converts text (LRC or plain) into lyrics.
localLyrics	3863	Lyrics saved on this device.
lyrSame	3871	Are two lyrics the same (text and timing)?
lyricsSaved	3879	Shows freshly saved lyrics until Jellyfin serves them.
getLyrics	3894	A song's lyrics (local, pending, cache or server).
setLyricsMode	3902	Opens or closes the lyrics with a transition.
setQueueMode	3915	Opens or closes the player's queue with a transition.
renderSheetQueue	3926	Queue list inside the player.
applyLyricsMode	3935	Applies lyrics mode without a transition.
reloadLyrics	3943	Reloads the current lyrics.
renderLyrics	3944	Draws the current song's lyrics.
estimateTimes	3968	Spreads approximate timing for unsynced lyrics.
syncLyrics	3975	Highlights the playing line and scrolls the lyrics.
refreshLyricsButtons	4011	Lyrics or Find lyrics button depending on whether there are lyrics.
syncStart	4026	Opens the editor to sync by hand.
syncEnd	4040	Closes the editor.
syncCancel	4044	Cancels syncing.
renderSync	4045	Draws the sync editor.
syncEditorTick	4052	Marks the playing line in the editor.
syncMark	4059	Marks the start of the highlighted line.
syncUndo	4072	Undoes the last mark.
syncNudge	4080	Moves the last mark ¼ s.
syncSave	4089	Fills in timing and saves the LRC.
saveLyricsText	4106	Saves lyrics in Jellyfin (or on the device if it can't).
lyricsSearch	4118	Searches LrcLib for lyrics and lets you choose one.
lyricsPaste	4148	Dialog to paste or edit lyrics.
forgetLocalLyrics	4166	Removes the lyrics saved on this device.

Queue from line 4169 · 4

Function	Line	What it does
renderQueue	4171	Draws the queue panel.
renderQueueAll	4180	Redraws every queue view.
moveQueue	4185	Moves a song in the queue (also in remote control).
markCurrent	4237	Highlights the playing song in lists.

Toasts, menus and dialogs from line 4240 · 2

Function	Line	What it does
toast	4242	Brief on-screen notice.
menu	4243	Options menu (bottom sheet).

My devices from line 4251 · 63

Function	Line	What it does
devLabel	4256	Friendly name of a device.
devName	4258	Device name for sentences.
expand6	4260	Expands an IPv6 address.
ipKey	4266	Network and privacy of an IP address.
sameNetwork	4274	Are two addresses on the same network?
RC.on	4286	Is My devices turned on?
RC.start	4287	Starts My devices (WebSocket and polling).
RC.stop	4288	Stops My devices.
RC.connect	4294	Opens Jellyfin's WebSocket with retries.
RC.send	4307	Sends a message over the WebSocket.
RC.onMessage	4308	Handles WebSocket messages (commands, favorites, keep-alive).
RC.fromMe	4319	Does the command come from your own user?
RC.onPlaystate	4326	Runs received pause, next, seek...
RC.onPlay	4339	Runs a play command (and adopts the DJ if it comes with it).
RC.onGeneral	4361	Runs received repeat or shuffle.
RC.queueForReport	4368	Queue reported to Jellyfin.
RC.reportSoon	4369	Reports the state to Jellyfin shortly.
RC.schedule	4375	Schedules the next device poll.
RC.poll	4376	Looks for your other Nanuqs and follows the one you control.
RC.maybeFollow	4399	Connects automatically to another Nanuq playing on your network.
RC.enter	4405	Starts controlling another device.
RC.leave	4421	Stops controlling and restores your queue.
RC.apply	4429	Mirrors here what's playing on the other device.
RC.pos	4460	Estimated second of the song on the other device.
RC.tick	4461	Updates bar, lyrics and system controls in remote control.
RC.decorate	4468	Shows “Playing on ...” in the player.
RC.fail	4474	Warns that a command couldn't be sent.

Function	Line	What it does
RC.cmd	4475	Sends a command to the playing device.
RC.seek	4481	Seeks on the other device.
RC.window	4482	Queue window that is sent (≤ 200).
RC.playQueue	4483	Sends a queue to play there.
RC.add	4492	Adds songs to the queue there.
RC.reorder	4502	Sends the reordered queue (keeping the DJ).
RC.general	4513	Sends a general command there.
RC.here	4515	“Play here”: brings the music (and the DJ) to this device.
RC.there	4538	Sends the music (and the DJ) to another device.
__nanuqTick	4557	20-second heartbeat from Android (WebSocket, party, stuck audio).
__nanuqVis	4565	Android reports whether the app is visible.
openDevices	4566	My devices menu.
updateFavBtn	4597	Player's favorite button.
setFavLocal	4603	Marks the favorite in the local queue.
favPending	4610	Remembers a recent favorite change.
favMerge	4611	Applies recent changes over the server's list.
favHold	4629	Protects the playing song's favorite (Jellyfin 12 bug).
favHeld	4632	Is this favorite being protected?
favReassert	4633	Saves the favorite again after each playback report.
favRemote	4641	Handles favorite changes made elsewhere.
refreshFav	4663	Confirms with the server whether the song is a favorite.
toggleFav	4674	Marks or unmarks the favorite and verifies it.
favConfirm	4694	Second confirmation against the favorites section.
updateDlBtn	4708	Player's Download button.
dlAddToast	4717	Notice when requesting a download.
dlTap	4726	Tap on Download (request or manage).
connTest	4739	“Test connection”: measures API, cover, audio, seeking and player.
startRadio	4867	Radio from a song.
addToPlaylistMenu	4881	Menu to add to a playlist (or create one).
songOptions	4896	Common song options.
queueMenu	4903	Menu for a queue song.
trackMenu	4916	Menu for a song in a list.

Function	Line	What it does
askConfirm	4924	Confirmation dialog.
askName	4930	Dialog to type a name.
openSheet	4938	Opens the full-screen player.
closeSheet	4946	Closes the full-screen player.

Party (watch party) from line 4949 · 65

Function	Line	What it does
partyHost	4956	Can this device host (desktop)?
partyGuest	4957	Is this device a guest (Android)?
partyOn	4958	Is the party available on this device?
partySongUrl	4960	MP3 320 kbps URL to send a song to the party.
partyCover	4966	240 px JPEG thumbnail of the cover.
partyCoverOk	4976	Does the cover come from the room (127.0.0.1 with token)?
partyClean	4979	Cleans the state sent by the computer (untrusted).
stopForParty	4991	Stops your music when opening the room.
PH.active	5006	Does the party handle the controls?
PH.init	5007	Prepares the party player and listens to main.js.
PH.onEv	5019	Handles room events (state, joined, left).
PH.apply	5025	Applies the room state (computer party mode).
PH.list	5037	Room song list.
PH.open	5038	Opens the room.
PH.close	5044	Closes the room.
PH.jump	5046	Plays a song from “Up next” right now.
PH.order	5052	Changes the list order.
PH.next	5054	Follows the list: plays the next ready song.
PH.play	5061	Plays a party song.
PH.stopCur	5069	Stops the current party song.
PH.done	5077	A song ended: deletes it and moves on.
PH.toggle	5078	Pauses or resumes the party.
PH.yieldTo	5091	You played your music: the party pauses and hands over the controls.
PH.media	5098	System controls with the party song.
PH.report	5110	Tells the room what's playing.
PH.loadLyrics	5115	Loads the party song's lyrics.
PH.line	5125	Current and next lyrics line.

Function	Line	What it does
PH.tick	5133	Karaoke: every second, time and line to the phones.
PG.connect	5143	Joins a room (with code or key) and reconnects on its own.
PG.leave	5170	Leaves the party.
PG.joined	5180	Are you at a party?
PG.ui	5181	Phone party mode (hides bars and player).
PG.wake	5190	The phone woke up: rejoins if needed.
PG.resume	5196	When opening the app, returns to the ongoing party.
PG.abortUp	5201	Cancels the upload in progress.
PG.onMsg	5202	Handles room messages.
PG.add	5229	Sends a song to the party (lyrics, cover and audio in chunks).
PG.remove	5285	Cancels a song that's still arriving.
PG.find	5287	Finds rooms on the network (UDP from Android).
partyAddr	5307	Converts the typed address into ws://.../fiesta.
iglooHTML	5318	Igloo HTML.
iglooBear	5334	A guest's Nanuq in their theme's costume.
iglooHoldSVG	5341	Cover held up with sparkles (who picked the song).
iglooHeldOnTop	5352	Puts the object the Nanuq was holding on top of the cover.
partyGuestColor	5363	Label color from the guest's theme.
partyCoverSrc	5371	Cover URL of a party song.
iglooSync	5379	Places guests in the circle without redrawing.
iglooLive	5413	Animates the igloo only while visible.
iglooQueue	5418	“Up next” row in full screen.
iglooQueueEdge	5430	Gradient if there are more songs out of view.
partyVolumeStep	5506	Volume with ↑ ↓ in full screen and its notice.
iglooIdle	5519	Hides the pointer after 2.5 s idle.
iglooVinyl	5529	Sleeve and vinyl record of the playing song.
iglooLine	5541	Lyrics on the ceiling with a sparkle.
renderPartySoon	5548	Redraws the party shortly.
partyPaintTick	5552	Updates the karaoke (line, next, time).
partyPaintUp	5558	Progress of the song you're sending.
partyRowsHTML	5559	Party song list.
partyGuestsHTML	5568	Guest list.
partyNowHTML	5569	What's playing at the party.
paintParty	5574	Draws the party screen (host or guest).

Function	Line	What it does
partySongsHTML	5635	Songs to send.
partySearch	5636	Party search (playlists, artists, albums, songs, lyrics).
partyBrowse	5664	Songs of an artist, album or playlist in the party.
V.party	5679	Party screen.

Global actions from line 5693 · 1

Function	Line	What it does
d0	5695	Shortcut for an element's dataset.

Login from line 5853 · 11

Function	Line	What it does
recentServers	5856	Servers used before.
rememberServer	5857	Remembers a server.
forgetServer	5863	Forgets a server.
probeTimed	5865	Tests a server measuring how long it takes.
canDiscover	5876	Can servers be discovered on the network?
discoverServers	5878	Discovers Jellyfin on the network (UDP 7359).
serverList	5897	Server list with their status.
cleartextRisk	5924	Would the password travel unencrypted over the internet?
showLogin	5935	Login screen.
startApp	6001	Starts the app with the session (queue, position, DJ, Home).
logout	6017	Logs out and deletes personal data.

17 Rest of the page

Languages and service worker.



web/i18n.js

1,024 lines · 1 functions

Function	Line	What it does
tx	1015	Translates a Spanish text into the chosen language and fills in the {0}, {1}... values.

web/sw.js

45 lines · 3 functions

Function	Line	What it does
install	8	Caches the app's files when the service worker installs.
activate	12	Deletes caches from older versions.
fetch	16	Serves only the app's files (network first for HTML/JS/CSS); the API and music go straight to the network.

18 Android

Native Java and the JavaScript bridge.



android/app/src/main/java/com/nanuq/app/Discovery.java

78 lines · 3 functions

Function	Line	What it does
`Discovery.Discovery`	27	Private constructor (static methods only).
`Discovery.find`	30	Discovers over UDP (Jellyfin or parties) and returns the answers as JSON.
`Discovery.send`	72	Sends the question to every broadcast address.

android/app/src/main/java/com/nanuq/app/Downloads.java

595 lines · 44 functions

Function	Line	What it does
`Downloads.get`	43	Single instance of downloads.
`Downloads.Downloads`	72	Sets up folder, index, local server and network watch.
`Downloads.audio`	83	Audio file for the local server.
`Downloads.audioMime`	84	Audio type for the local server.
`Downloads.cover`	85	Cover for the local server.
`Downloads.prefetch`	86	Ahead-of-time song for the local server.
`Downloads.prefetchMime`	87	Type of the ahead-of-time song.
`Downloads.onAvailable`	91	Network is back: retries downloads.
`Downloads.onCapabilitiesChanged`	92	Network changed: checks the download queue.
`Downloads.setListener`	97	Connects the page to receive changes.
`Downloads.configure`	101	Maximum songs and Wi-Fi only.
`Downloads.base`	117	Local server address.
`Downloads.okUrl`	127	Is the URL shaped like a Jellyfin audio or cover?
`Downloads.openSameHost`	141	Opens a connection following redirects only to the same server.
`Downloads.enqueue`	162	Adds a download (or updates it).
`Downloads.setScore`	196	Updates how many times a download played.
`Downloads.remove`	202	Removes a download.
`Downloads.removeAll`	208	Removes all (or only automatic ones).
`Downloads.list`	214	List of downloads as JSON.

Function	Line	What it does
`Downloads.network`	222	Current network (wifi, cellular, none), looking beneath a VPN.
`Downloads.underlying`	236	Real network beneath a VPN.
`Downloads.file`	245	File of a ready download.
`Downloads.coverFile`	253	A download's cover.
`Downloads.setCover`	262	Requests an old download's cover.
`Downloads.fetchCoverLater`	270	Downloads the cover in the background.
`Downloads.fetchCover`	273	Downloads the cover (≤ 15 MB).
`Downloads.mime`	301	A download's audio type.
`Downloads.retryAndKick`	309	Retries failed ones and continues.
`Downloads.kick`	316	Starts the download queue.
`Downloads.kickLocked`	318	Starts the queue (already locked).
`Downloads.loop`	324	Downloads pending ones one at a time.
`Downloads.nextPending`	344	The next one (yours first).
`Downloads.download`	354	Downloads a song with limits and free space; Opus only if complete (OggCheck).
`Downloads.victim`	431	The least-played automatic one (to make room).
`Downloads.removeEntry`	440	Deletes a download and its files.
`Downloads.caps`	454	Active network capabilities.
`Downloads.unmetered`	459	Is the network unmetered (Wi-Fi, Ethernet)?
`Downloads.networkAllowed`	464	Can it download on this network?
`Downloads.safe`	472	Safe file name for an id.
`Downloads.toJson`	474	Download as JSON for the page.
`Downloads.emit`	492	Notifies the page of a change.
`Downloads.ensureLoaded`	498	Reads the index when the store opens.
`Downloads.load`	500	Reads the encrypted index (or recovers from files).
`Downloads.save`	566	Saves the encrypted index.

`android/app/src/main/java/com/nanuq/app/LocalServer.java` 184 lines · 7 functions

Function	Line	What it does
`LocalServer.LocalServer`	46	Opens the server on 127.0.0.1 with a random token.
`LocalServer.base`	60	Base address with the token.
`LocalServer.acceptLoop`	62	Accepts connections (max. 32 at a time).
`LocalServer.readLine`	74	Reads a request line with a cap and a deadline.
`LocalServer.handle`	86	Handles a request (10 s and ≤ 100 headers).
`LocalServer.status`	129	Answers a status code.

Function	Line	What it does
`LocalServer.serveFile`	135	Serves a file with ranges (seeking within the song).

android/app/src/main/java/com/nanuq/app/MainActivity.java

406 lines · 44 functions

Function	Line	What it does
`MainActivity.run`	63	Heartbeat every 20 s to the page.
`MainActivity.onCreate`	70	Creates the WebView, connects services and loads the app.
`MainActivity.createWeb`	88	Configures the WebView (no file access, bridge).
`MainActivity.recreateWeb`	114	Recreates the WebView if the system killed it.
`MainActivity.dispatchMedia`	124	Passes a notification button to the page.
`MainActivity.onBackPressed`	135	Back: the page decides or the app goes to the background.
`MainActivity.onStart`	144	The app came back into view.
`MainActivity.onStop`	147	The app went to the background (saves the store).
`MainActivity.onDestroy`	150	Releases the WebView and the notification.
`MainActivity.setBarColor`	164	System bar color according to the theme.
`MainActivity.showNotice`	185	“Requested song ready” notification.
`MainActivity.setMetadata`	213	Bridge: title, artist, album and cover.
`MainActivity.setPlaybackState`	214	Bridge: playing or paused.
`MainActivity.setActions`	215	Bridge: available buttons.
`MainActivity.setPosition`	216	Bridge: position and duration.
`MainActivity.dlList`	224	Bridge: download list.
`MainActivity.dlEnqueue`	225	Bridge: download a song.
`MainActivity.dlCover`	226	Bridge: request a cover.
`MainActivity.dlBase`	227	Bridge: local server address.
`MainActivity.dlRemove`	228	Bridge: remove a download.
`MainActivity.dlRemoveAll`	229	Bridge: remove downloads.
`MainActivity.dlConfig`	230	Bridge: download settings.
`MainActivity.dlScore`	231	Bridge: plays of a download.
`MainActivity.dlNetwork`	232	Bridge: network type.
`MainActivity.pfSet`	235	Bridge: ahead-of-time window.
`MainActivity.pfReady`	236	Bridge: is that song already saved?
`MainActivity.pfBroken`	237	Bridge: that saved song didn't play.
`MainActivity.http`	240	Bridge: CORS-free request to SomeDL.
`MainActivity.secKind`	248	Bridge: encrypted store state.
`MainActivity.secGet`	249	Bridge: read an encrypted value.

Function	Line	What it does
`MainActivity.secSet`	250	Bridge: save an encrypted value.
`MainActivity.secDel`	251	Bridge: delete an encrypted value.
`MainActivity.secReset`	253	Bridge: start over.
`MainActivity.videoBegin`	257	Bridge: starts saving a video (MP4/WebM).
`MainActivity.videoChunk`	269	Bridge: a chunk of the video (≤ 300 MB).
`MainActivity.videoEnd`	281	Bridge: finishes and saves in Movies › Nanuq.
`MainActivity.videoShare`	323	Bridge: share the video just saved.
`MainActivity.notice`	332	Bridge: system notification.
`MainActivity.partyDiscover`	335	Bridge: find parties over UDP.
`MainActivity.discover`	343	Bridge: find Jellyfin over UDP.
`MainActivity.onRenderProcessGone`	354	The WebView process died: it's recreated.
`MainActivity.shouldInterceptRequest`	360	Serves the app from the assets at http://localhost.
`MainActivity.shouldOverrideUrlLoading`	377	Only http(s) links go out to the browser.
`MainActivity.mime`	399	Type of an asset file.

`android/app/src/main/java/com/nanuq/app/MediaCenter.java` 372 lines · 33 functions

Function	Line	What it does
`MediaCenter.get`	52	Single instance of the media center.
`MediaCenter.MediaCenter`	72	Creates the MediaSession and the notification channel.
`MediaCenter.onPlay`	83	System play button.
`MediaCenter.onPause`	84	System pause button.
`MediaCenter.onStop`	85	System stop button.
`MediaCenter.onSkipToNext`	86	System next button.
`MediaCenter.onSkipToPrevious`	87	System previous button.
`MediaCenter.onSeekTo`	88	Seek from the system.
`MediaCenter.onFastForward`	89	Fast-forward from the system.
`MediaCenter.onRewind`	90	Rewind from the system.
`MediaCenter.attach`	96	Connects the page.
`MediaCenter.detach`	98	Disconnects the page.
`MediaCenter.hasPage`	101	Is there a page to receive commands?
`MediaCenter.stopIfNoPage`	104	Without a page, removes the notification.
`MediaCenter.dispatch`	113	Sends an action to the page.
`MediaCenter.setMetadata`	120	Song details (and loads the cover).
`MediaCenter.setPlaybackState`	141	Playback state.

Function	Line	What it does
`MediaCenter.setActions`	152	Available buttons.
`MediaCenter.setPosition`	160	Position and duration.
`MediaCenter.isPlaying`	174	Is it playing?
`MediaCenter.currentPosition`	176	Estimated current position.
`MediaCenter.publishMetadata`	181	Publishes the details on the MediaSession.
`MediaCenter.publishState`	191	Publishes the state and buttons.
`MediaCenter.refresh`	212	Adjusts service, notification and locks.
`MediaCenter.shutdown`	240	Stops everything when the app closes.
`MediaCenter.buildNotification`	251	Builds the playback notification.
`MediaCenter.action`	280	A notification button.
`MediaCenter.serviceIntent`	284	A button's intent to the service.
`MediaCenter.contentIntent`	292	Open the app from the notification.
`MediaCenter.onReceive`	300	Headphones unplugged: pause.
`MediaCenter.setNoisyReceiver`	305	Listens for headphone disconnection.
`MediaCenter.loadBitmap`	319	Loads the cover with limits (8 MB, reduced).
`MediaCenter.readCapped`	364	Reads with a byte cap.

`android/app/src/main/java/com/nanuq/app/NativeHttp.java` 84 lines · 5 functions

Function	Line	What it does
`NativeHttp.NativeHttp`	19	Private constructor (static methods only).
`NativeHttp.Result`	24	Response (status and text).
`NativeHttp.allowed`	30	Allowed SomeDL route?
`NativeHttp.fetch`	37	CORS-free request only to SomeDL, no redirects.
`NativeHttp.readAll`	71	Reads the response (≤ 4 MB).

`android/app/src/main/java/com/nanuq/app/PlaybackService.java` 101 lines · 7 functions

Function	Line	What it does
`PlaybackService.onCreate`	31	Prepares the CPU and Wi-Fi locks.
`PlaybackService.onStartCommand`	46	Foreground and notification buttons.
`PlaybackService.setLocks`	65	Takes or releases the locks.
`PlaybackService.release`	76	Leaves the foreground and ends.
`PlaybackService.onTaskRemoved`	84	Removed from recents: closes everything.
`PlaybackService.onDestroy`	92	Releases the locks.
`PlaybackService.onBind`	99	No binding (started service).

android/app/src/main/java/com/nanuq/app/Prefetch.java 222 lines · 16 functions

Function	Line	What it does
Prefetch.get	40	Single instance of the ahead-of-time cache.
Prefetch.Prefetch	61	Empties the previous cache and watches the network.
Prefetch.onAvailable	71	Network is back: keeps downloading.
Prefetch.set	80	Receives the desired window and deletes what's left over.
Prefetch.broken	109	A saved one didn't play: deleted and not repeated.
Prefetch.ready	116	Is that song ready?
Prefetch.file	118	File of a ready song.
Prefetch.mime	125	Audio type.
Prefetch.drop	130	Removes a song and its file.
Prefetch.deleteFiles	132	Deletes a song's files.
Prefetch.kick	139	Starts the downloads.
Prefetch.kickLocked	141	Starts (already locked).
Prefetch.loop	147	Downloads pending ones one at a time (respects the total cap).
Prefetch.online	160	Is there a network?
Prefetch.fetch	166	Downloads a song with limits; Opus only if complete (OggCheck).
Prefetch.trim	211	If over the total cap, drops the farthest ones.

android/app/src/main/java/com/nanuq/app/SecureStore.java 192 lines · 16 functions

Function	Line	What it does
SecureStore.get	44	Single instance (or reads a value if given a key).
SecureStore.SecureStore	58	Opens the store when created.
SecureStore.open	69	Opens key (Keystore) and file: ready, locked or lost.
SecureStore.kind	104	State (retries if it was locked).
SecureStore.reset	113	Start over: sets the file aside and creates a new key.
SecureStore.ready	122	Is it open?
SecureStore.set	129	Saves a value (≤ 8 MB).
SecureStore.del	136	Deletes a value.
SecureStore.validKey	142	Valid key?
SecureStore.scheduleSave	144	Saves in 400 ms.
SecureStore.flush	150	Saves now.
SecureStore.saveNow	152	Encrypts and writes the file.
SecureStore.encrypt	161	Encrypts with AES-256-GCM.

Function	Line	What it does
SecureStore.decrypt	171	Decrypts and verifies.
SecureStore.readAll	178	Reads a file (≤ 64 MB).
SecureStore.writeAtomic	186	Writes atomically.

android/bridge/android-bridge.js 97 lines · 9 functions

Function	Line	What it does
abs	17	Turns a relative URL into an absolute one.
MediaMetadata	19	MediaMetadata replacement for the page.
bestArt	28	Chooses the best cover (≤ 512 px) for the notification.
mediaSession.metadata	42	Sends title, artist, album and cover to Android.
mediaSession.playbackState	47	Sends the playback state to Android.
mediaSession.setActionHandler	54	Registers the buttons the page handles.
mediaSession.setPositionState	58	Sends the position only when it drifts.
__nanuqMedia	76	Runs a notification or headphone button on the page.
sendTheme	87	Sends the theme color to the system bars.

android/build-apk.sh 19 lines · 1 functions

Function	Line	What it does
(script)	1	Builds the APK and leaves it in dist/<version>/.

android/app/src/main/java/com/nanuq/app/OggCheck.java 76 lines · 2 functions

Function	Line	What it does
OggCheck.valid	42	Checks that an Ogg (Opus) file has all its pages complete, with correct CRC and end of stream.
OggCheck.mime	70	Accepts the server's audio type only in the audio/... form.

19 Desktop

Electron and its packaging.



linux/main.js

855 lines · 61 functions

Function	Line	What it does
serve	47	Hardened internal 127.0.0.1:47863 server that hands the app to the window (and the party).
broadcastOf	79	Broadcast address of an IPv4 network.
discover	80	Discovers Jellyfin servers over UDP 7359.
fromApp	100	Does the IPC message come from the app's page?
vaultBackend	132	Which system keyring is available (GNOME Keyring, KWallet, DPAPI, Keychain).
secFile	148	Path of the encrypted data file.
secKeyFile	149	Path of the data key encrypted with the keyring.
writeAtomic	150	Writes a file atomically (0600).
secInit	153	Opens the encrypted store (AES-256-GCM): locked, lost or ready.
secFlush	190	Writes the encrypted store to disk.
dcSet	228	Records the Discord connection state.
dcPaths	229	Possible Discord sockets or pipes (only yours).
dcWrite	240	Sends a message in Discord's format.
dcRetry	241	Looks for Discord again in 10 s.
dcConnect	242	Connects to Discord trying each socket and greets it.
dcData	258	Reads Discord's messages with limits (READY, ERROR, close, ping).
dcFlush	281	Sends the activity respecting Discord's rate (and the minimal one if rejected).
dcActivity	292	Builds the clean, bounded “Listening” activity.
jparse	329	Parses outside JSON without toString/valueOf/__proto__.
ptStr	330	Converts to safe, bounded text (no control chars).
ptIp	331	IP address without the ::ffff: prefix.
ptLocal	335	Is the address on your local network?
ptOpenNow	342	Is the room open?

Function	Line	What it does
ptEmit	343	Notifies the page of a room event.
ptBase	348	Folder on disk where the room's songs are stored (on Linux, ~/.cache/nanuq; it used to be the temp folder, which is RAM in the Flatpak).
ptCleanTemp	353	Deletes temporary folders from earlier rooms.
ptNonce	366	AES-GCM nonce for a block: the song's 8 random bytes + the block number.
ptSeal	367	Encrypts and authenticates a 64 KB block of a party song with the room key.
ptEncSize	372	Exact size the encrypted file must have (detects truncated or extended files).
ptOpenRange	373	Reads, checks and decrypts only the blocks of a requested range; fails if a block was altered.
ptSniff	389	Is it audio? (by its first bytes).
ptCover	402	Validates the cover a phone sends (JPEG/PNG/WebP ≤ 150 KB).
ptPublic	411	Public data of a song in the list.
ptGuests	415	Present and away guests.
ptAddrs	416	This computer's IPv4 addresses.
ptStatus	421	Full room state for the page.
ptState	427	Broadcasts the state to everyone (batched in 150 ms).
ptStateNow	428	Sends the state now.
ptKill	434	Removes a song from the list and the disk.
ptFrame	443	Writes a WebSocket frame.
ptSend	450	Sends a JSON message to a phone.
ptDrop	451	Closes a phone's connection (with a reason).
ptGone	459	A phone disconnected (stays away or left).
ptUpgrade	470	Accepts the WebSocket connection (local network only, limits).
ptData	490	Reads WebSocket frames with every validation.
ptMsg	523	Handles a message from a phone.
ptNewCode	536	Changes the room code.
ptHello	540	A phone joining: code, key, brute force.
ptAdd	570	A phone announces a song (with limits).
ptChunk	588	Receives a piece of the song and writes it to disk.
ptEnd	606	The song finished arriving: checks it's audio.

Function	Line	What it does
ptServe	623	Serves the room's songs and covers to the window (with Range).
ptGuestCover	651	Serves covers to the phones (only with their key).
ptGc	660	Cleanup every 30 s (old, half-arrived, away, silent).
ptListen	670	Opens the room port (47864 or a free one).
ptOpen	683	Opens the room: server, UDP, code and token.
ptClose	710	Closes the room and cuts every connection.
stateFile	770	File with the window's size and position.
loadState	772	Reads the window's size and position.
saveState	777	Saves the window's size and position.
start	783	Starts the app: store, server, window, permissions and protections.

linux/preload.js

25 lines · 6 functions

Function	Line	What it does
nanuqDesktop.discover	7	Bridge: discover Jellyfin on the network.
nanuqDesktop.http	8	Bridge: CORS-free request to SomeDL.
nanuqDesktop.discord	11	Bridge: Discord activity (or its status).
nanuqDesktop.party.op	14	Bridge: party operations (open, close, jump, order, karaoke...).
nanuqDesktop.party.on	15	Bridge: listen to room events.
nanuqDesktop.secure.*	17	Synchronous bridge to the encrypted store (kind, get, set, del, reset).

linux/prepare-web.js

16 lines · 1 functions

Function	Line	What it does
(script)	3	Copies web/ to www/ and takes the version from app.js.

linux/collect.js

15 lines · 1 functions

Function	Line	What it does
(script)	3	Moves the built packages to dist/<version>/.

linux/flatpak/build-flatpak.sh

54 lines · 1 functions

Function	Line	What it does
(script)	1	Builds the Flatpak, publishes it to the signed local repository and installs or updates it.

20 Build

The script that builds everything.



scripts/build.sh

68 lines · 1 functions

Function	Line	What it does
(script)	1	Bumps the version and builds Android, Linux, Flatpak and Windows.

Made with care (and lots of ice)

If you change something, remember: the interface lives in web/ and every text goes through tx().

